

**INSTITUTO FEDERAL DE EDUCAÇÃO,
CIÊNCIA E TECNOLOGIA**
RIO GRANDE DO NORTE
Campus Pau dos Ferros

Primeira entrega do Projeto Integrador
FlowSLA

Gabriel Enzo
Luis Felipe
Rangel Gomes Rocha de Oliveira

Pau dos Ferros/RN
2026

1. Visão Geral

1.1. Problema, Justificativa e Público-Alvo

Problema

Prestadoras de serviço técnico que operam sob contrato B2B com manutenção predial, refrigeração, assistência de equipamentos, infraestrutura de rede, entre outras, assumem, junto aos seus clientes, prazos formais de atendimento (Service Level Agreement, SLA): tempo para responder à solicitação, tempo para comparecer ao local e tempo para resolver o problema. O cumprimento desses prazos é, muitas vezes, o próprio diferencial competitivo do serviço prestado.

Na prática observada em empresas de pequeno e médio porte desse setor, o controle desses prazos é feito de forma manual, por meio de planilhas, e-mail e aplicativos de mensagens instantâneas. Essa abordagem operacional apresenta três limitações centrais:

1. **Ausência de detecção automática de violação.** Sem um mecanismo de apuração contínuo, o descumprimento de um prazo contratual só é percebido quando o cliente formaliza uma reclamação, momento em que o dano à relação comercial já ocorreu.
2. **Fragilidade na comprovação do atendimento.** Registros fotográficos ou em vídeo do serviço executado permanecem dispersos em dispositivos pessoais dos técnicos ou em conversas informais, sem vínculo formal ao chamado correspondente e sem garantia de integridade.
3. **Inexistência de histórico auditável.** Diante de um questionamento do cliente sobre o cumprimento do contrato, a prestadora frequentemente não dispõe de um registro estruturado e confiável que demonstre o que foi executado, quando e por qual profissional.

Justificativa

A ausência de automação no controle de SLA gera risco direto ao negócio da prestadora: perda de credibilidade contratual, dificuldade de renegociação de contratos e exposição em eventuais disputas comerciais. Um sistema que automatize o cálculo dos prazos, detecte violações no momento em que ocorrem e mantenha evidências de atendimento íntegras e rastreáveis resolve uma necessidade operacional real desse segmento, ao mesmo tempo

em que reúne as características que justificam o desenvolvimento de um sistema corporativo: múltiplos perfis de usuário com permissões distintas, regras de negócio não triviais, um processo de alto custo computacional que justifica execução assíncrona, e dados que demandam trilha de auditoria.

Público-alvo

Prestadoras de serviço técnico de pequeno e médio porte que atendem múltiplos clientes corporativos sob contrato, possuem equipe de atendimento em campo e têm a necessidade formal de comprovar o cumprimento de prazos acordados.

1.2. Objetivos do Sistema e Escopo

Objetivo geral

Desenvolver uma plataforma corporativa para gestão de chamados técnicos sob contrato de SLA, capaz de calcular prazos automaticamente a partir de políticas contratadas, detectar violações sem intervenção humana, e registrar evidências de atendimento de forma íntegra, auditável e assíncrona.

Objetivos específicos

- Automatizar o cálculo de prazos contratuais a partir de políticas de SLA parametrizáveis, considerando janela de atendimento e nível de prioridade.
- Detectar, em tempo hábil, o descumprimento de prazos de resposta, atendimento em campo e resolução.
- Permitir o registro de evidências técnicas (imagem e vídeo) vinculadas ao atendimento, com processamento assíncrono que não penaliza a experiência do usuário.
- Assegurar rastreabilidade das operações sensíveis do sistema por meio de trilha de auditoria.
- Demonstrar, por meio de teste de carga, a capacidade de escalonamento horizontal do componente de processamento de evidências.

Escopo

O sistema contempla:

- Cadastro de clientes, contratos e políticas de SLA.
- Abertura, triagem, atribuição e acompanhamento de chamados técnicos.
- Registro de atendimento em campo com anexação de evidências (imagem e vídeo).
- Cálculo automático de três indicadores de SLA independentes: resposta, atendimento em campo e resolução.
- Detecção automática de violação de prazo e validação automática de chamados.
- Geração de relatório gerencial de cumprimento de SLA.
- Trilha de auditoria das operações sensíveis.
- Política de retenção e expurgo automático de evidências e de registros de auditoria.

O que está fora do escopo

Item	Justificativa da exclusão
Faturamento e cobrança	O sistema trata da execução do serviço, não da relação financeira com o cliente.
Integração com ERP	Fora do domínio de controle de SLA; tratada como possível evolução futura.
Aplicativo mobile nativo	Substituído, nesta versão, por frontend web responsivo; mantido como evolução futura.
Geolocalização do técnico em campo	Decisão de minimização de dados pessoais; mantida como evolução futura.

Item	Justificativa da exclusão
Watermark de evidências	Substituído pelo cálculo de hash criptográfico do arquivo original, que cumpre o mesmo papel de garantia de integridade com menor complexidade e menor risco de exposição de dados pessoais.
Fase de pausa do relógio de SLA ("aguardando cliente")	Simplificação deliberada do modelo de contagem de tempo; limitação registrada e tratada como evolução futura.
Gestão de estoque de peças	Fora do domínio de controle de prazo contratual.
Roteirização e otimização de deslocamento	Fora do domínio de controle de prazo contratual.

1.3. Perfis de Usuário e Permissões

O sistema define três perfis de usuário, com autenticação e autorização geridas por provedor de identidade externo (Keycloak), garantindo que o controle de acesso seja aplicado por papel na fronteira da API e, adicionalmente, por isolamento de dados na camada de serviço.

Cliente

Representa o contato da empresa contratante do serviço. Pode:

- Abrir chamados vinculados a um contrato vigente, sugerindo a prioridade inicial.
- Consultar exclusivamente os chamados vinculados aos contratos da própria empresa.

- Acompanhar o status e os prazos de cada chamado.
- Validar a conclusão de um atendimento, encerrando o chamado.
- Reabrir um chamado resolvido, mediante justificativa obrigatória.
- Visualizar as evidências anexadas ao próprio chamado.
- Receber notificação por e-mail nos marcos relevantes do atendimento.

Não possui acesso a dados de outros clientes, a valores comerciais do contrato, nem à trilha de auditoria interna.

Técnico

Representa o profissional de campo responsável pela execução presencial do serviço.

Pode:

- Consultar exclusivamente os chamados a ele atribuídos.
- Registrar o início e a execução do atendimento presencial.
- Anexar evidências fotográficas ou em vídeo ao atendimento.
- Consultar o status de processamento das evidências enviadas.
- Concluir o atendimento, mediante a existência de ao menos uma evidência registrada.

Não possui acesso a dados comerciais do contrato nem a chamados de outros técnicos.

Gestor

Representa o coordenador de operações da prestadora de serviço. Pode:

- Cadastrar e manter clientes, contratos e políticas de SLA.
- Realizar a triagem do chamado, definindo sua prioridade definitiva.
- Registrar o primeiro posicionamento ao cliente.
- Atribuir e/ou reatribuir chamados a técnicos.
- Cancelar chamados, mediante motivo obrigatório.
- Solicitar o reprocessamento de evidências com falha.
- Gerar e exportar relatórios gerenciais de cumprimento de SLA.
- Consultar a trilha de auditoria interna do sistema.

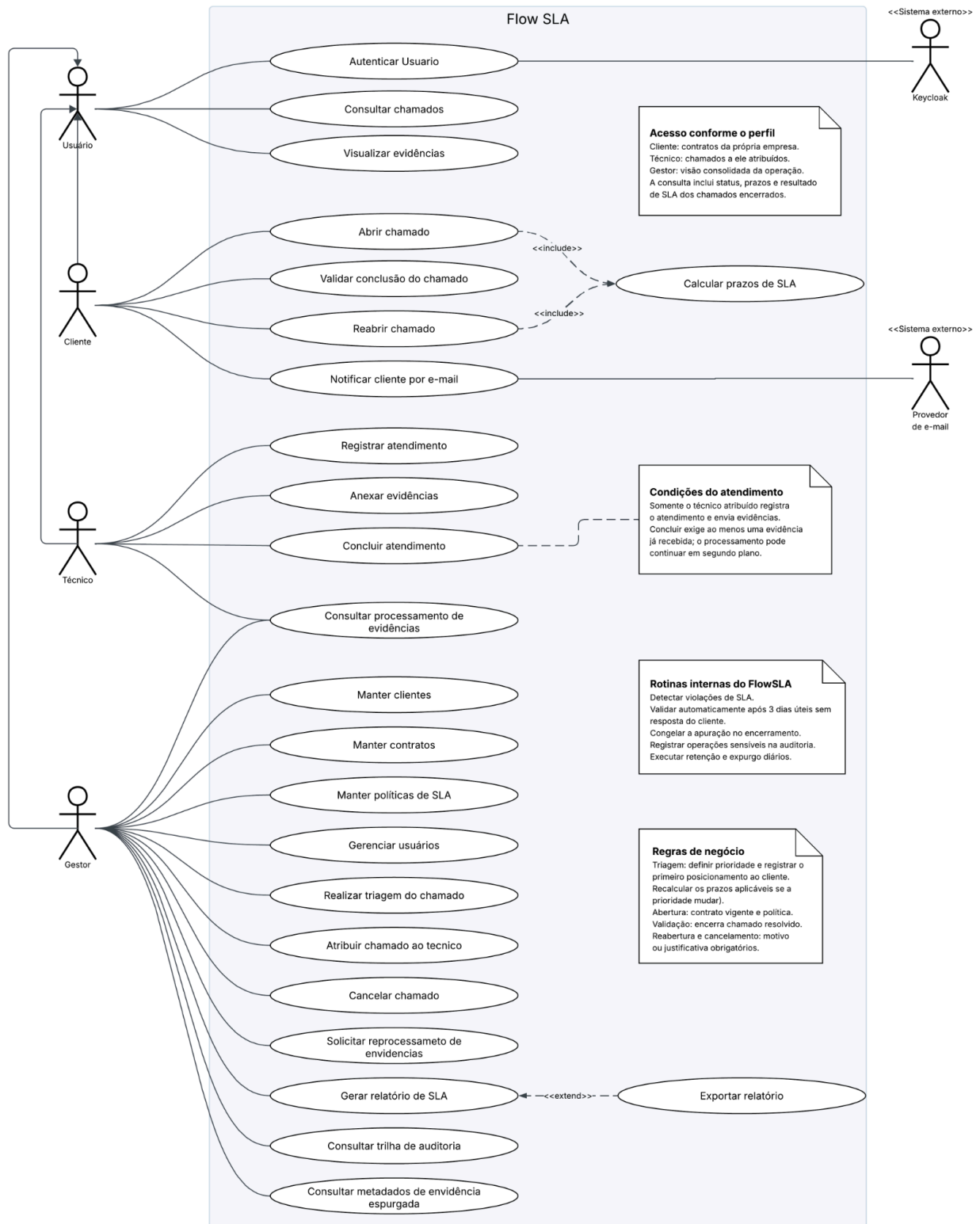
É o único perfil com visão consolidada sobre todos os clientes, contratos e chamados do sistema.

Matriz-síntese de permissões

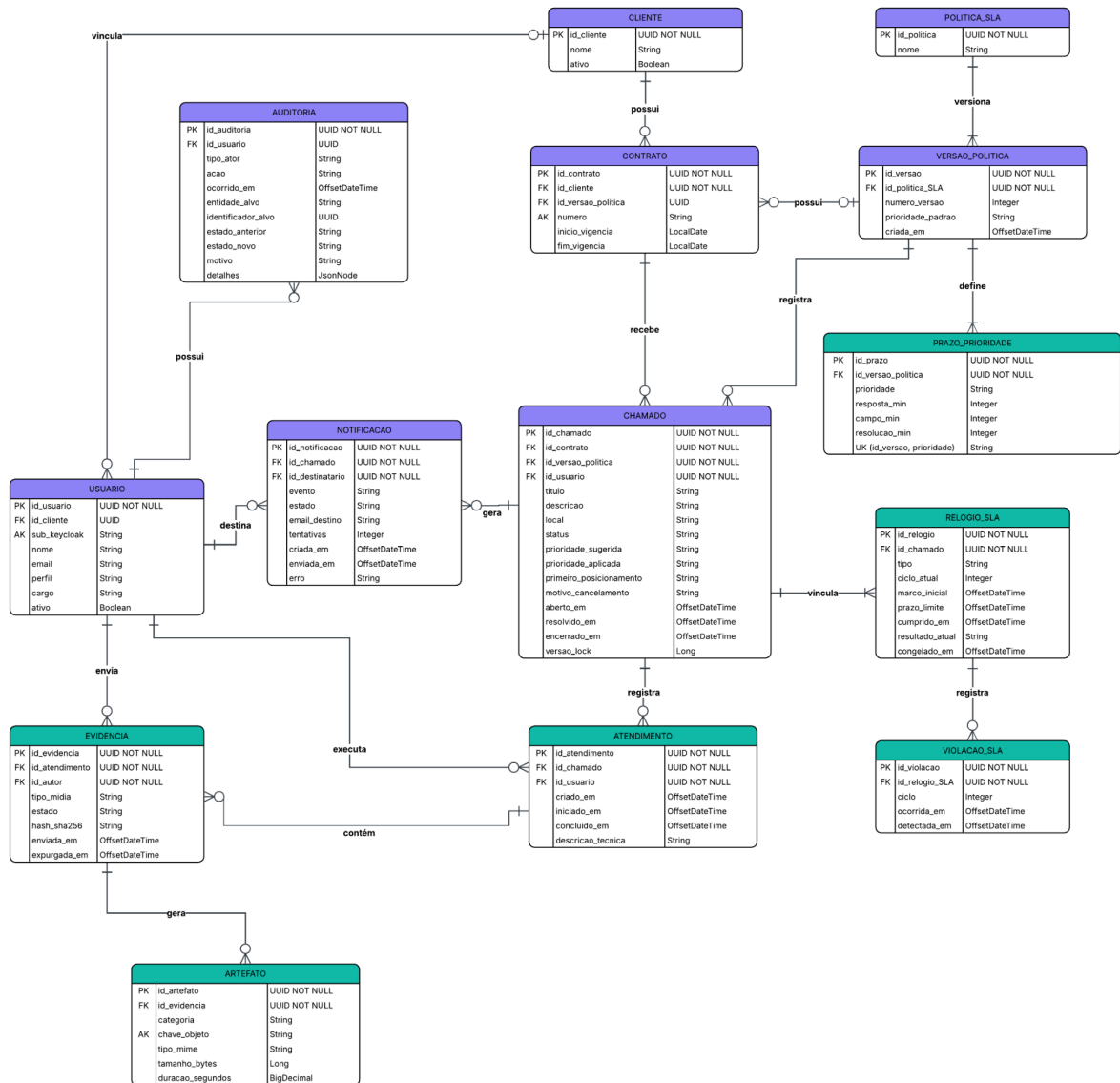
Ação	Cliente	Técnico	Gestor
Abrir chamado	✓	-	-
Definir prioridade definitiva	-	-	✓
Atribuir chamado	-	-	✓
Registrar atendimento e evidências	-	✓	-
Validar / Reabrir chamado	✓	-	-
Consultar dados de outros clientes	-	-	✓
Consultar trilha de auditoria	-	-	✓
Gerar relatório gerencial	-	-	✓

2. Modelagem

2.1. Diagrama de caso de uso

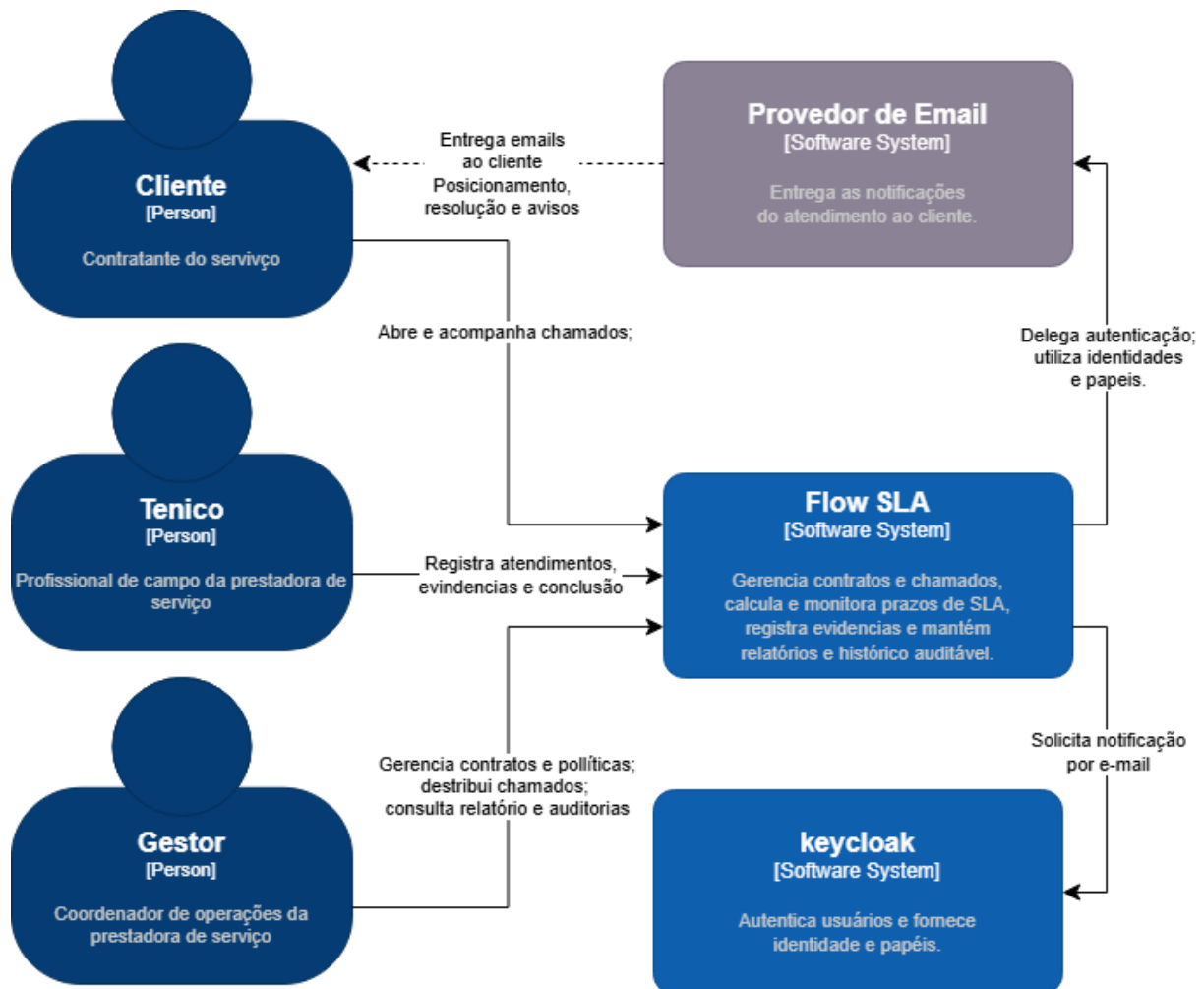


2.2. Diagrama entidade relacionamento

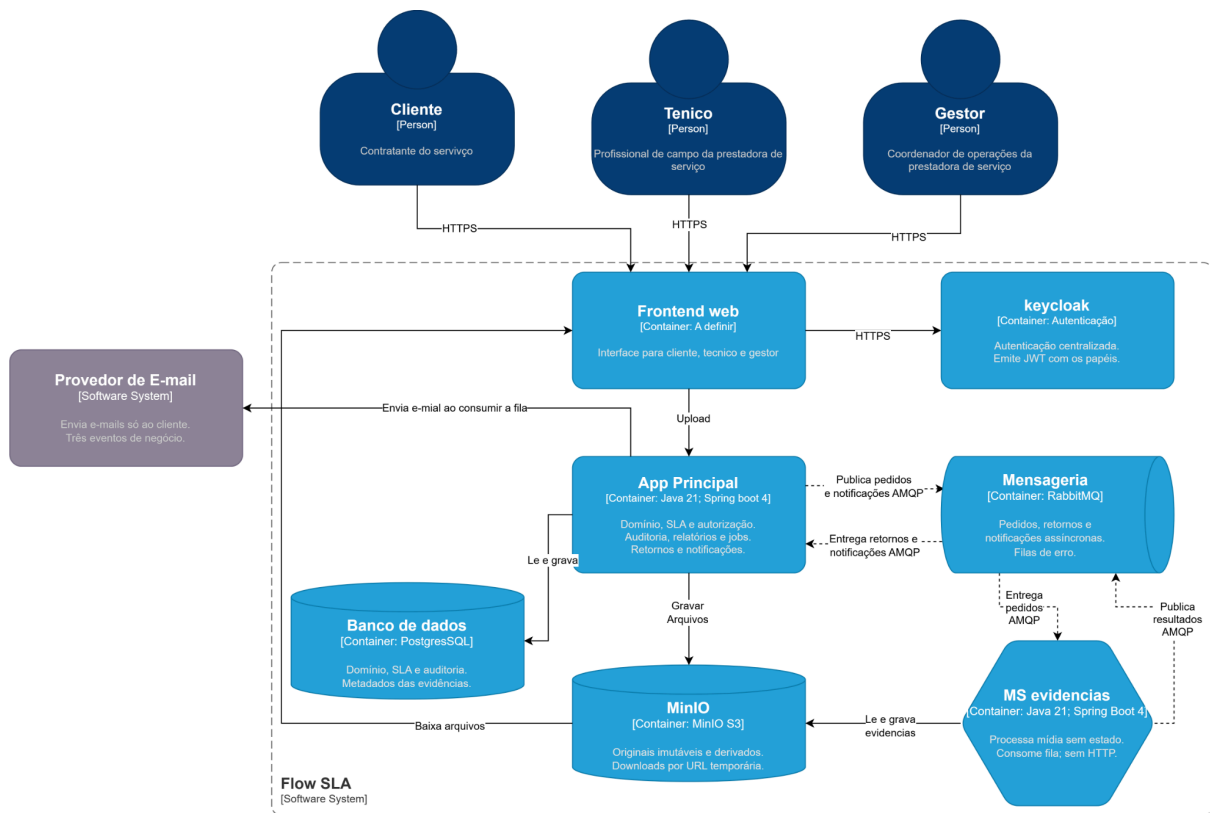


2.4. Diagrama de arquitetura do sistema

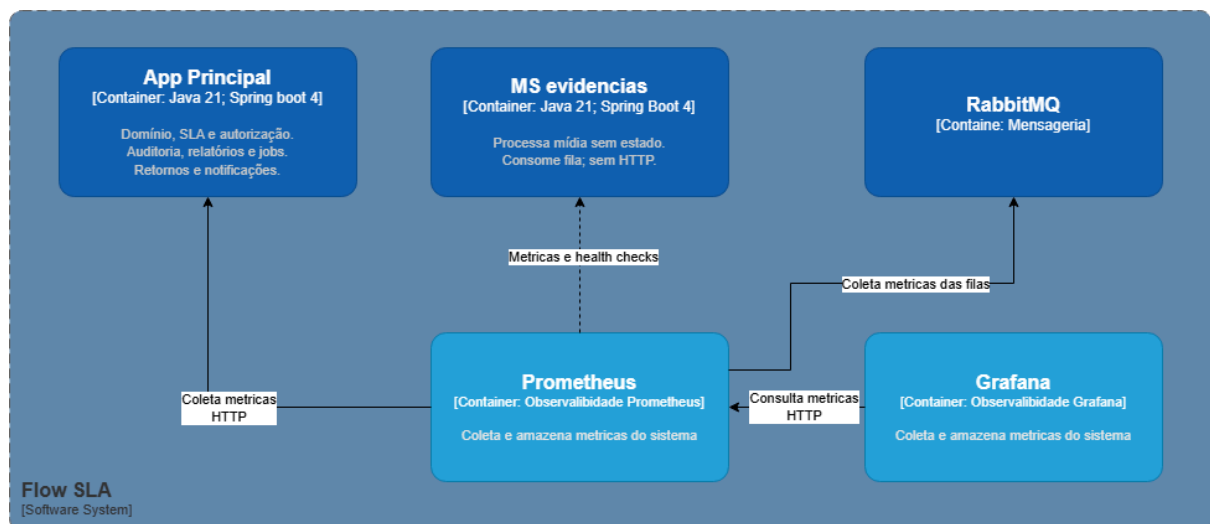
2.4.1. Modelo C4 nível 1



2.4.2. Modelo C4 nível 2 - Aplicação



2.4.3. Modelo C4 nível 2 - MS



3. Decisões Técnicas

3.1. Stack Tecnológica e Justificativa de Cada Escolha

A stack adotada segue integralmente o conjunto de tecnologias obrigatórias definido pela disciplina, com as escolhas específicas justificadas abaixo.

Núcleo da aplicação

Tecnologia	Justificativa
Java 21 (LTS) + Spring Boot	Exigência da disciplina e escolha ideal por ser uma plataforma corporativa madura e extremamente robusta, garantindo a segurança esperada para o FlowSLA. O ecossistema Spring acelera o desenvolvimento ao fornecer bibliotecas prontas para uso e suporte nativo integrado para persistência, segurança, mensageria e observabilidade.
PostgreSQL	Banco relacional robusto para os dados transacionais do domínio, garantindo controle de concorrência e integridade referencial. Destaca-se por permitir operações complexas e oferecer suporte nativo a campos JSONB , um recurso estratégico para o armazenamento de logs complexos e do histórico dinâmico de atendimentos dos SLAs.
Spring Data JPA / Hibernate	Camada de persistência produtiva, com controle explícito de transações via @Transactional .

Tecnologia	Justificativa
Flyway	Versionamento de schema rastreável; substitui a geração automática de schema, inadequada a um sistema corporativo.
springdoc-openapi (Swagger)	Documentação de API gerada a partir do código, mantendo-a sempre sincronizada com a implementação.

Serviços de apoio

Tecnologia	Justificativa
Keycloak	Provedor de identidade externo (OAuth 2.1 / OIDC), desacoplando autenticação do código da aplicação e permitindo gestão centralizada de papéis.
RabbitMQ	Escolhido em vez de Kafka por o caso de uso do projeto ser fila de trabalho com roteamento e <i>dead letter queue</i> nativa, e não streaming de eventos com necessidade de replay. RabbitMQ atende ao requisito com menor complexidade operacional.
MinIO (compatível com S3)	Armazenamento de objetos para as evidências. Um banco relacional não é adequado para arquivos binários de grande volume: infla o backup e degrada o desempenho de consulta. A compatibilidade com a API

Tecnologia	Justificativa
	S3 permite migração futura para um provedor de nuvem sem alteração de código.
Resilience4j	Implementação de padrões de resiliência (timeout, retry, circuit breaker) nas chamadas externas do sistema.
Prometheus + Grafana	Coleta de métricas e painel de observabilidade, integrados ao Spring Boot Actuator e Micrometer.

Qualidade e entrega

Tecnologia	Justificativa
JUnit 5 + Mockito	Ferramentas padrão para testes no ecossistema Spring, oferecendo 100% de integração direta sem necessidade de configurações complexas. Serão utilizadas para a criação de testes unitários rápidos e isolados de dependências externas, com foco principal em garantir a exatidão da lógica de cálculo de SLA.
Testcontainers	Permite que os testes de integração possuam determinadas configurações em cima de imagens Docker limpas (como uma imagem Postgres, por

Tecnologia	Justificativa
	exemplo) e mantém o ciclo de vida diretamente ligado aos testes.
k6	Ferramenta de teste de carga escolhida por permitir script versionado em JavaScript e execução em contêiner, alinhada à necessidade de comprovar a escalabilidade do microsserviço.
Docker + Docker Compose	O empacotamento em contêineres é uma exigência do projeto para garantir a total reprodutibilidade do sistema e simplificar o processo de deploy. A combinação com o Docker Compose permite orquestrar todos os serviços dependentes de forma ágil, gerando três ambientes isolados (desenvolvimento, testes e produção) facilmente executáveis em qualquer máquina.
GitHub Actions	Ferramenta escolhida por sua facilidade de configuração e integração nativa com o repositório do projeto.

3.2. Microsserviço: Definição, Existência e Necessidade de Escala Independente

Qual é

O microsserviço **ms-evidencias** é responsável pelo processamento das evidências técnicas (imagens e vídeos) anexadas pelos técnicos durante o atendimento em campo. Suas responsabilidades são:

- Transcodificação de vídeo para formato e resolução padronizados.
- Geração de derivados de imagem (thumbnail e versões redimensionadas).

- Remoção de metadados EXIF, por decisão de minimização de dados pessoais.
- Cálculo de hash criptográfico dos arquivos processados, garantindo integridade.

Por que ele existe

O processamento de mídia é a única operação do sistema com custo de CPU relevante: a transcodificação de um vídeo de até 30 segundos consome tempo mensurável em segundos, ordem de grandeza incompatível com o tempo de resposta esperado de uma API REST. Caso essa responsabilidade permanecesse dentro da aplicação principal, cada envio de evidência bloquearia uma *thread* da aplicação pelo tempo total do processamento, comprometendo a capacidade de atendimento a todos os demais usuários do sistema durante picos de uso.

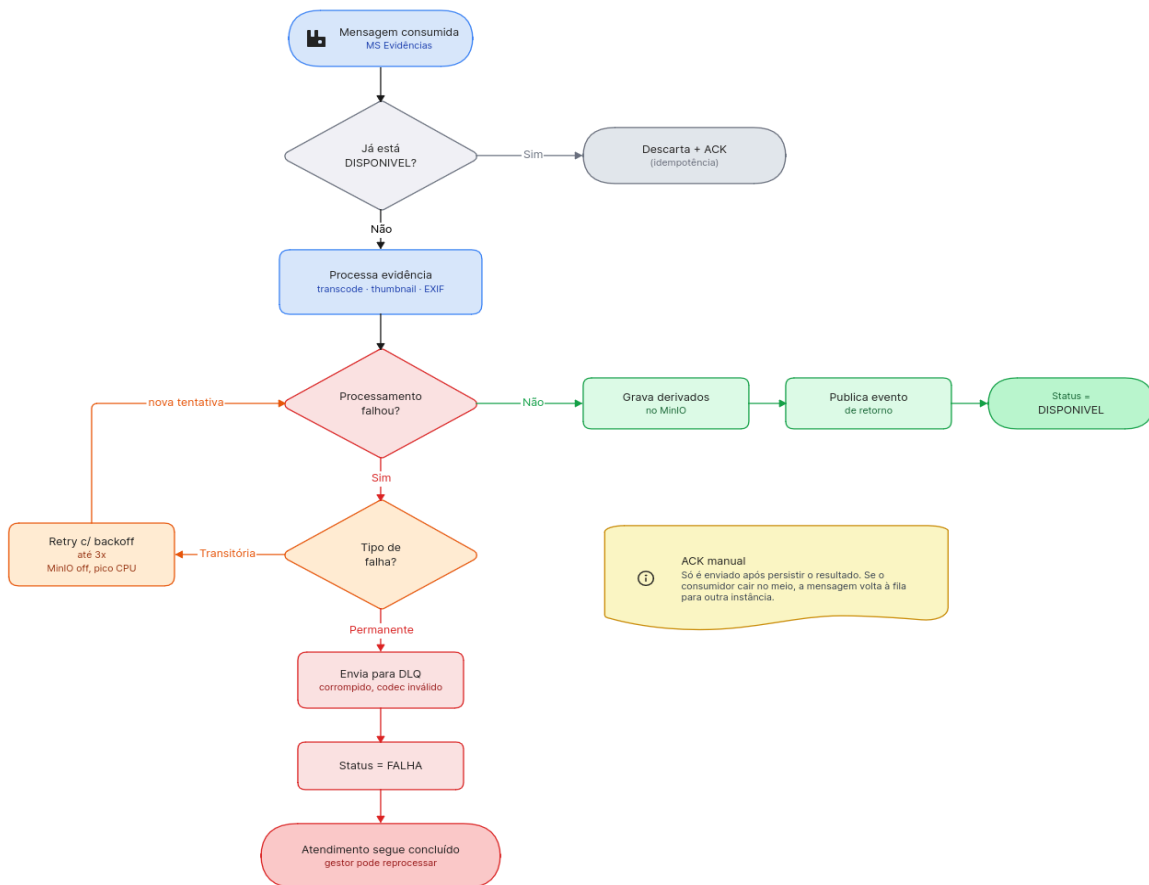
Por que precisa escalar separadamente

O **ms-evidencias** concentra um perfil de carga distinto do restante da aplicação: picos concentrados de uso intensivo de CPU, correlacionados a horários de maior atividade de campo, enquanto o restante do sistema mantém carga majoritariamente leve e constante (consultas, cadastros, apuração de SLA). Ele apresenta três propriedades que o tornam adequado ao escalonamento horizontal independente:

- **Não realiza autenticação ou autorização:** Essas responsabilidades já foram aplicadas pela aplicação principal no momento da recepção da evidência.
- **É desprovido de estado:** Cada mensagem consumida é autocontida, o que torna qualquer instância intercambiável com as demais.
- **Não expõe porta HTTP:** A sua replicação não exige balanceador de carga nem gerenciamento de sessão, apenas o acréscimo de novos consumidores à mesma fila.

Essa separação permite escalar exclusivamente o componente sob pressão, sem replicar o restante da aplicação, e será comprovada por meio de teste de carga comparando o comportamento do sistema com uma e com múltiplas instâncias do microserviço.

3.3. Desenho do Fluxo de Mensageria



3.4. Estratégia de Autenticação e Autorização

A autenticação do sistema é delegada a um provedor de identidade externo (Keycloak), seguindo o protocolo OAuth 2.1 / OpenID Connect. A aplicação principal e o microserviço de evidências operam como *resource servers* que validam tokens JWT emitidos pelo Keycloak, sem necessidade de consulta síncrona ao provedor a cada requisição.

A autorização é aplicada em dois níveis complementares:

Autorização por papel. Cada endpoint da API declara explicitamente os papéis autorizados a acessá-lo, refletindo os três perfis definidos (Cliente, Técnico, Gestor). Essa verificação ocorre na camada de segurança da aplicação, antes de qualquer lógica de negócio ser executada.

Autorização por dado. Além da verificação de papel, o sistema aplica um filtro de propriedade dos dados na camada de serviço: um usuário do perfil Cliente somente acessa registros vinculados aos contratos de sua própria empresa; um usuário do perfil Técnico somente acessa os chamados a ele atribuídos. Esse filtro é aplicado de forma independente

da requisição recebida, nunca a partir de um parâmetro informado pelo cliente HTTP, prevenindo que a alteração de um identificador na requisição resulte em acesso indevido a dados de terceiros.

Segredos de configuração (credenciais de banco de dados, chaves de integração) são mantidos fora do código-fonte, externalizados por variáveis de ambiente, e nenhum segredo é versionado no repositório.

3.5. Dados Pessoais Tratados e Implicações de LGPD

Dados pessoais tratados pelo sistema

Dado	Titular	Finalidade	Base legal	Retenção
Nome e e-mail corporativo	Usuários dos três perfis	Identificação, acesso e comunicação	Execução de contrato	Vigência do contrato, acrescida de prazo legal
Função ou cargo	Técnico e Gestor	Atribuição de chamados e definição de permissões	Execução de contrato	Idem
Conteúdo das evidências (imagem e vídeo)	Cliente e seus equipamentos	Comprovação da execução do serviço contratado	Execução de contrato	180 dias após o encerramento do chamado
Registros da trilha de auditoria	Usuários dos três perfis	Rastreabilidade e responsabilização sobre operações do sistema	Legítimo interesse	5 anos a partir do evento registrado

Ausência de dados sensíveis

O sistema não realiza tratamento de dados pessoais sensíveis, na acepção do art. 5º, inciso II, da Lei Geral de Proteção de Dados (saúde, biometria, origem racial ou étnica, entre outros). Essa característica decorre de duas decisões de minimização adotadas deliberadamente no desenho do sistema:

- **Não há coleta de geolocalização** do técnico em campo, tampouco nos metadados dos arquivos enviados.
- **Os metadados EXIF são removidos** de toda evidência durante o processamento assíncrono, eliminando qualquer coordenada de geolocalização eventualmente embutida pelo dispositivo de captura, ainda que de forma não intencional.

Política de retenção e expurgo

Os prazos de retenção definidos (180 dias para evidências e 5 anos para registros de auditoria) constituem decisão de negócio e arquitetura do projeto, e não uma exigência textual da LGPD, pois a legislação exige a existência de uma política de retenção coerente com a finalidade do tratamento, sem determinar prazos específicos.

O prazo de 180 dias para as evidências busca equilibrar a necessidade de comprovação da execução do serviço e de eventual contestação posterior pelo cliente com o princípio da minimização de dados e a redução do consumo de armazenamento. Encerrado o prazo, uma rotina automática executa o expurgo do conteúdo armazenado, preservando apenas metadados mínimos de rastreabilidade. Nenhum perfil do sistema possui operação manual para exclusão de evidências, o que garante que a retenção seja cumprida de forma consistente e não sujeita a decisão individual.

O prazo de 5 anos para os registros de auditoria reflete sua finalidade distinta que são a rastreabilidade e responsabilização, não comprovação de serviços, seu volume de armazenamento significativamente inferior ao das evidências em imagem e vídeo. O registro de auditoria é imutável durante todo o período de retenção, distinção conceitual relevante: imutabilidade descreve a impossibilidade de alteração enquanto o registro existe, e não a permanência indefinida do dado.

4. Organização

4.1. Estrutura de Repositórios e Orquestração

Atualmente, a arquitetura do projeto está distribuída em dois repositórios iniciais: um destinado ao backend principal e outro exclusivo para a documentação. O planejamento arquitetural prevê a criação de repositórios adicionais para o frontend e para o microsserviço de processamento de evidências.

O empacotamento em contêineres é uma exigência do projeto para garantir a total reprodutibilidade do sistema e simplificar o processo de deploy. Por isso, o repositório do backend já conta com um `Dockerfile` configurado com a técnica de *multistage building*. Essa abordagem foi adotada com o intuito de otimizar a construção, tornando a imagem final mais leve e facilitando a gestão dos ambientes.

A combinação com o Docker Compose permite orquestrar todos os serviços dependentes de forma ágil, gerando três ambientes isolados (desenvolvimento, testes e produção) facilmente executáveis em qualquer máquina. Para garantir o isolamento máximo e a segurança, foram criados arquivos Docker Compose distintos para cada ambiente. Esses arquivos estendem e sobrescrevem as configurações de um `docker-compose.yml` base, reduzindo a duplicação de código. Além disso, as credenciais e variáveis sensíveis são injetadas de forma segura através de arquivos `.env` localizados em um diretório oculto (`.envs`).

[Dockerfile](#) e [Docker Compose base](#) no GitHub

4.2. Divisão de Responsabilidades (Matriz RACI Adaptada)

Para garantir uma gestão de projeto eficiente e definir de forma clara os papéis de cada integrante da equipe, adotou-se uma metodologia baseada na adaptação da Matriz RACI. Essa gestão foi dividida em duas perspectivas complementares: macro e micro.

- **Visão Macro:** Está diretamente atrelada à estrutura de repositórios e domínios do projeto. Nesta camada, define-se a responsabilidade principal de cada integrante (por exemplo, liderança em Frontend, Backend, Microsserviço ou Documentação). Detalhes específicos sobre a alocação de domínios podem ser consultados no [repositório de documentação](#) da equipe.
- **Visão Micro:** Foca na granularidade das atividades diárias. Cada tarefa ou *issue* mapeada exige a designação clara do papel dos integrantes envolvidos em sua execução. Essa flexibilidade permite que um membro da equipe seja designado para atuar em uma tarefa específica fora de seu domínio principal, promovendo colaboração e garantindo que os gargalos de desenvolvimento sejam mitigados.

4.3. Pipeline de Integração Contínua (CI)

A ferramenta GitHub Actions foi escolhida por sua facilidade de configuração e integração nativa com o repositório do projeto.

O projeto conta com uma esteira de Integração Contínua (CI) configurada no repositório de backend, que é acionada automaticamente a cada Pull Request submetido ou *push* direcionado às branches principais (**main** e **develop**). A pipeline executa automaticamente o build, os testes automatizados e uma análise estática a cada Pull Request criado pelos desenvolvedores. Esse processo garante a qualidade e a integridade do código, barrando falhas antes que sejam integradas à base principal do sistema.

[Arquivo ci.yml no GitHub](#)

5. Testes

5.1. Plano de testes

Plano de Testes
Nome do Projeto: FlowSLA - Plataforma Corporativa de Controle de SLAs e Evidências
Pessoas envolvidas / Responsabilidades
Desenvolvedor(a) Backend - Criação e execução dos testes unitários (JUnit 5) e de integração (Testcontainers)
Desenvolvedor(a) Frontend - Criação e execução dos testes unitários/integração (Vitest) e testes ponta a ponta (Playwright)
Responsável de DevOps/Infraestrutura – Criação e execução dos testes de carga (k6) e orquestração do CI/CD no repositório
Funcionalidades ou Módulos
Gestão de SLAs e processamento assíncrono de mídias. Autenticação delegada via Keycloak (OAuth 2.1). Autorização em dois níveis: por papéis (Cliente, Técnico e Gestor) e por propriedade de dado. Conformidade com LGPD (remoção de metadados EXIF) e externalização estrita de segredos de configuração.
Equipamentos / Softwares
O sistema deve funcionar em navegadores web modernos para desktops e dispositivos móveis (homologado via Chromium, Firefox e WebKit). O backend e o microserviço devem ser executados de forma empacotada em contêineres Docker.
Cronograma
Data de Início e Fim do Teste: 10/08/2026 - 18/12/2026
Local de Testes
Executados nas máquinas locais dos desenvolvedores e na pipeline do GitHub Actions, divididos por camadas:
- Testes Unitários: Executados em memória, sem necessidade de uma infraestrutura;
- Testes de Integração: Executados sob instâncias efêmeras orquestradas via Testcontainers;
- Testes E2E: Executados em um ambiente de testes completo orquestrado via Docker Compose.
Critério para considerar o teste finalizado
O teste será considerado finalizado e aprovado mediante dupla validação: execução com sucesso na pipeline automatizada (GitHub Actions) e aprovação humana obrigatória via Code Review no Pull Request, realizada estritamente pelo Aprovador (A) definido na issue, garantindo que o executor não valide o próprio trabalho.
Observações
Notificações de falhas são disparadas automaticamente pelo GitHub Actions assim que a pipeline identificar quebras no código. Os testes do microserviço focarão especificamente na validação do consumo de CPU durante operações assíncronas.

5.2. Casos de testes iniciais

ID	Cenário de Teste	Categoria	Ferramenta	Componente Alvo	Resultado Esperado
TS-001	Acesso a rota protegida com JWT inválido, expirado ou ausente	Teste unitário	JUnit 5 + Mockito	Backend principal	A camada de segurança do Spring Boot (Resource Server) rejeita a requisição imediatamente, retornando o código HTTP 401 (Unauthorized) antes de atingir qualquer lógica de negócio.
TS-002	Validação de autorização por papel (RBAC) via payload do JWT	Teste unitário	JUnit 5 + Mockito	Backend principal	A API retorna HTTP 403 (Forbidden) caso receba um JWT com assinatura válida, mas cujo <i>payload</i> indique o papel de 'Cliente' tentando acessar um endpoint exclusivo para o perfil 'Gestor'.

TS-003	Validação da assinatura do token gerado pelo Keycloak	Teste de integração	Testcontainers	Backend principal	Utilizando um contêiner efêmero para simular o provedor OAuth 2.1, o backend valida criptograficamente a assinatura do token gerado e permite o acesso ao recurso (HTTP 200 OK).
TS-004	Comportamento da interface (Guards) em um ambiente de cookie seguro	Teste unitário	Vitest	Frontend (Angular)	Como o token é mantido em um cookie HttpOnly inatingível pelo JavaScript, o teste valida se o Angular reage corretamente ao estado de autenticação gerido pelo navegador, ocultando menus ou bloqueando rotas não autorizadas na interface.